

Competitive Programming

Section 68: Advanced Tree Decompositions

Competitive Programming Course

Table of contents

1	One tree, four notions of compression	3
2	Heavy-light decomposition with direction	4
3	Centroid ancestor paths	5
4	Virtual trees	6
5	Small-to-large merging	7
6	Worked trace: direction in heavy-light composition	8
7	Worked trace: centroid inclusion and subtraction	9
8	Worked trace: one virtual edge counts many pairs	10
9	Recognition table	11
10	C++ and Python implementation notes	12
11	Proof habits	13
12	Common mistakes	14
13	Exercises	15
14	Before submitting	16

1 One tree, four notions of compression

- Heavy-light decomposition turns a path into logarithmically many array intervals.
- Centroid decomposition gives every vertex logarithmically many balanced separator ancestors.
- A virtual tree keeps only query vertices and LCAs needed to connect them.
- Small-to-large merging moves each map entry only logarithmically many times.

The shared principle is to preserve exactly the boundary information a query needs while discarding most original edges from the active computation.

2 Heavy-light decomposition with direction

Choose the largest child subtree as heavy. Every root-to-vertex path crosses at most $O(\log n)$ light edges, because a light child's subtree is at most half its parent's. Consecutive heavy edges form an interval in the HLD order.

For sums, interval direction did not matter. Function composition and matrix multiplication are noncommutative. A segment node must store:

```
forward = product/composition from left to right
backward = product/composition from right to left
```

When querying $u \rightarrow v$, accumulate upward chunks from u at the end of a left answer and prepend downward chunks from v to a right answer. Join them only after both vertices share a chain.

3 Centroid ancestor paths

A centroid leaves no remaining component larger than half. Remove it and recurse. Decomposition depth is $O(\log n)$.

For every original vertex u , store $(\text{centroid}, \text{distance}(u, \text{centroid}))$ for all centroid ancestors. If each centroid knows the minimum distance from itself to an active vertex, then

```
nearest(u) = min(distance(u, c) + best_active_distance[c]).
```

For sums, direct addition over centroid ancestors double-counts active vertices that share the same centroid branch. Store totals for each centroid and subtotals for every branch; add the former and subtract the branch containing the query vertex.

4 Virtual trees

For k marked vertices, sort by Euler entry time and add LCAs of consecutive vertices. Sort and deduplicate again. The result has at most $2k-1$ vertices.

Use a stack in Euler order. Pop until its top is an ancestor of the new vertex; that top is the new vertex's virtual parent. A virtual edge (p, u) represents the entire original path of length $\text{dist}[u] - \text{dist}[p]$.

For pairwise marked distances, if $\text{cnt}[u]$ marked vertices lie below virtual edge (p, u) , exactly $\text{cnt}[u] * (k - \text{cnt}[u])$ unordered pairs use that edge.

5 Small-to-large merging

Suppose every subtree needs a color-frequency map. Keep the largest child map as the base and insert all entries from smaller maps. Whenever an entry moves, its destination map is at least twice the size of its previous map. It moves at most $O(\log n)$ times, giving $O(n \log n)$ expected work with hash maps.

Track the requested aggregate while counts increase; do not rescan the full map at every vertex.

6 Worked trace: direction in heavy-light composition

Let the vertex functions on a path 1-2-3 be applied when visiting the vertices:

```
f1(x) = x + 1  
f2(x) = 2x  
f3(x) = x + 4
```

Travelling 1 → 3 applies **f1**, then **f2**, then **f3**, so zero becomes 6. Travelling 3 → 1 applies **f3**, then **f2**, then **f1**, so zero becomes 9. The set of vertices is identical; only the order changed. Therefore a segment node storing only one aggregate cannot answer both directions.

For adjacent array intervals L then R, use the following convention:

```
forward(parent) = forward(R) after forward(L)  
backward(parent) = backward(L) after backward(R)
```

Write the convention in words before translating it into operator syntax. Whether source code spells composition as **left * right** or **right * left** depends on the chosen representation; copying a formula without its convention is a frequent source of reversed answers.

7 Worked trace: centroid inclusion and subtraction

Suppose removing centroid c creates branches A , B , and C . Query vertex u lies in A . A total stored at c contains active vertices from all three branches. For an active vertex x in B or C , the path $u..x$ passes through c , so

$$\text{dist}(u,x) = \text{dist}(u,c) + \text{dist}(c,x).$$

For x in branch A , that equality can overestimate because the path may remain entirely inside A . The query at c therefore adds the global total and subtracts the subtotal belonging to A . At a lower centroid inside A , u and x eventually fall into different branches; that is the one level where x is added and not subtracted.

This gives a useful proof template for centroid sums:

Charge each target vertex to the first centroid ancestor that separates it from the query vertex.

8 Worked trace: one virtual edge counts many pairs

Assume a virtual edge has original-tree length 5. Three marked vertices are below it and four are outside. Exactly $3 \cdot 4 = 12$ unordered marked pairs cross the edge. Each such path contains all five compressed original edges, so this virtual edge contributes $5 \cdot 12 = 60$ to the sum of pairwise distances.

The formula does not enumerate pairs. It changes the counting order:

```
sum over marked pairs of path length  
= sum over tree edges of number of marked pairs using that edge.
```

Whenever a problem asks for a sum over many paths, try this edge-contribution reversal before reaching for a more complicated data structure.

9 Recognition table

Statement or constraint shape	Candidate decomposition
online path queries and point/range updates	HLD
toggle vertices and query distances globally	centroid decomposition
each query names only k important tree vertices	virtual tree
every subtree needs a mergeable frequency summary	small-to-large

These are not interchangeable speed tricks. They preserve different boundaries: ordered path intervals, balanced separators, query-local Steiner structure, or amortized containers.

10 C++ and Python implementation notes

For HLD, keep `parent`, `depth`, `heavy`, `head`, and `position` in separate arrays and test the position layout before adding the segment tree. In Python, iterative parent/order construction avoids recursion-depth failures.

For centroid decomposition, C++ multisets make deletion direct. Python usually uses an insertion heap plus a deletion heap and removes equal tops lazily. Stale heap elements are harmless only if cleanup runs before reading the minimum.

Virtual-tree work arrays should be reset only on vertices used by the current query. Clearing $O(n)$ state per query destroys the intended complexity. Small-to-large code should return or retain the chosen largest container; copying it accidentally loses the amortized bound.

11 Proof habits

- For HLD, prove both that there are logarithmically many chunks and that their concatenation order matches the requested direction.
- For centroid queries, prove that every target is counted at least once and then identify the unique level where it survives subtraction.
- For virtual trees, prove that consecutive-Euler LCAs include every branching point required to preserve paths between marked vertices.
- For small-to-large, attach the complexity argument to one map entry: every move at least doubles the size of its container.

12 Common mistakes

- Treating noncommutative HLD chunks as if reversing did nothing.
- Proving few heavy chains but building positions that do not keep chains contiguous.
- Storing centroid totals without branch subtraction for sum queries.
- Omitting LCAs when constructing a virtual tree.
- Reusing stale per-query virtual counts.
- Merging the largest map into a smaller one.

13 Exercises

A and B isolate direction and balanced separators. C is blind and requires a query-sized tree. D makes the amortized small-to-large proof operational.

14 Before submitting

Test chains, stars, balanced trees, $u=v$, one marked vertex, ancestor-marked pairs, repeated toggles, and all colors equal or distinct.